画像処理実験 第 5 回

学生番号: 09B24490
加藤弘都 (KATO, HIROTO)

2026 年 10 月 23 日

1　課題 5

1.1　[5-2]

1.1.1　「特徴点らしさ」の大きいものを MAX 個選び出す

まず，特徴点の座標と特徴点らしさを格納した配列 ary の特徴点らしさの要素を降順にソートして先頭から MAX=30 個の特徴点を選ぶ．以下はその処理を表すコードである．ソートには np.argsort 関数を用いて，ary[:, 2] で配列の第三要素（特徴点らしさ）について昇順にソートし，[::-1] で配列を降順にしている．

```
1:    ary = ary[np.argsort(ary[:, 2])[::-1]]
2:    ary=ary[:MAX]
```

このとき，図??のような画像になった．画像をみると，左側のもとの画像より特徴点の個数が減っており，特徴点を 30 個選んだことがわかる．

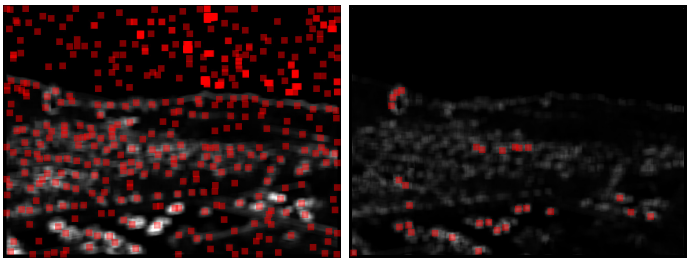


図 1　「特徴らしさ」をすべて出した画像（左）と大きいものを 30 個選んだ画像（右）

1.2　[5-2]

1.2.1　LocalMax 関数の構造

LocalMax 関数は以下のようなコードである．

```
1:    for v in range(-W,W+1):
2:     for u in range(-W,W+1):
3:      out[W:-W,W:-W]=np.maximum(out[W:-W,W:-W],
4:                                src[v+W:src.shape[0]+v-W,
5:                                    u+W:src.shape[1]+u-W])
```

この関数では，out が u，v によって変化しない画像で，u，v によって位置が変化する画像と np.maximum で比較する．画像同士の対応する各画素について比較し，大きい方の値を out の対応する場所に格納する．

つまりこの関数では以下のような数式で，u，v の変動範囲で各画素の近傍での最大値検出している．

$D(x,y) = \max_{u,v} I(x+u, y+v)$

1.2.2 LocalMax 関数を縦横 2 段階に分解

先程の近傍での最大値を検出する式を以下のように二段階二分解して，計算の効率化を図る．

$$O(x,y) = \frac{1}{15}\sum_{v=-7}^{7} T(x,y+v) \tag{1}$$

$$\text{where} \quad T(x,y) = \frac{1}{15}\sum_{u=-7}^{7} I(x+u,y) \tag{2}$$

この式をコードにすると以下のようになる．

```
 1: def LocalMax(src,W=7):
 2:   out=np.zeros_like(src)
 3:   t=np.zeros_like(src)
 4:   for v in range(-W,W +1):
 5:     t[W:-W,:]=np.maximum(t[W:-W,:],
 6:                          src[v+W:v+src.shape[0]-W,:])
 7:   for u in range(-W,W+1):
 8:     out[:,W:-W]=np.maximum(out[:,W:-W],
 9:                            t[:,u+W:u+t.shape[1]-W])
10:   return out
```

表 1　変更前後の時間の変化

	変更前	変更後
LocalMax 関数でかかった時間	0.206	0.035

このコードへの変更前後でのかかる時間の変化を表**??**に示す．この表をみると，変更後のコードのほうが早くなっていることが分かり，処理を効率化できている．

変更前のコードは二重ループを使用しているため $(2W+1)*(2W+1)*$(画素数) 回計算しているが，変更後は $(2W+1)*$(画素数)$*2$ 回の計算するため，処理を効率化できたと考える．

1.2.3 NAIVE と速度・結果を比較

表 2　変更前後の時間の変化

	FAST	NAIVE
極大判定でかかった時間	0.004	0.164

表**??**は NAIVE との速度を比較した結果である．これを見ると，NAIVE の方が遅くなっていることがわかる．NAIVE では二重ループしているため，FAST よりも計算量が増えたのではないかと考える．

2　感想

参考文献